AI-Assisted Programming in Software Engineering: A Systematic Literature Review on Capabilities, Productivity, and Code Quality

L.F. Guevara-Chávez
School of Engineering and Science
Tecnológico de Monterrey
Tampico, México
leon.guevara@tec.mx

***Abstract*—The rapid adoption of AI-assisted programming tools, particularly those based on large language models (LLMs), has significantly transformed software development practices. While these tools promise substantial productivity gains, existing evidence remains fragmented and, in some cases, contradictory. This study presents a systematic literature review of empirical research published between 2020 and 2026, synthesizing findings from controlled experiments, large-scale observational studies, industrial deployments, and qualitative investigations.**

The analysis integrates evidence across multiple dimensions, including productivity, code quality, technical debt, human–AI interaction, cognitive load, organizational impact, economic viability, and developer perception. Results indicate that AI-assisted programming consistently increases development output; however, these gains are often accompanied by trade-offs, such as higher verification effort, increased technical debt, and shifts in maintenance responsibilities. Furthermore, the effectiveness of these tools is highly contingent on factors such as developer experience, intensity of adoption, and the quality of human–AI interaction, particularly in iterative prompting and validation processes.

The findings also reveal a notable gap between perceived and actual productivity, as well as the importance of cognitive and sociotechnical factors, including trust calibration and community-driven heuristics, in shaping tool usage. From an economic perspective, AI-assisted development can be viable under appropriate conditions, although it introduces additional costs related to training, supervision, and integration.

Overall, this study advances the understanding of AI-assisted programming as a multifaceted, socio-technical phenomenon. Rather than acting as a simple productivity enhancer, these tools reshape how developers work, collaborate, and make decisions, highlighting the need for balanced adoption strategies and further longitudinal research.

I. Introduction

The rapid advancement of large language models (LLMs) has significantly transformed software development practices, particularly through the emergence of AI-assisted code generation tools. Systems such as GitHub Copilot and ChatGPT are increasingly integrated into development workflows, enabling developers to generate, complete, and refactor code using natural language prompts.

Recent empirical evidence suggests that these tools can substantially impact developer productivity. For instance, controlled experiments and large-scale observational studies report measurable improvements in coding speed and task completion efficiency when developers use AI-assisted tools [1]. Similarly, empirical evaluations of GitHub Copilot suggest that AI-generated code can assist developers by providing correct or partially correct solutions in a significant proportion of cases, potentially reducing the effort required to complete programming tasks, although effectiveness varies across programming languages and problem instances [2].

Despite these benefits, concerns have been raised regarding the quality and security of AI-generated code. Prior work has shown that AI-assisted tools may introduce security vulnerabilities in generated code, highlighting the need for careful human review and security-aware development practices [3]. In addition, comparative evaluations of multiple AI coding assistants highlight variability in correctness and maintainability across different tools and scenarios, with some studies also examining efficiency-related metrics [4], [5].

Although the number of studies in this area has grown rapidly, the literature remains fragmented. Existing research varies significantly in terms of methodologies, evaluation metrics, and tools analyzed, making it difficult to draw consistent conclusions. Furthermore, most studies focus on a limited subset of tools, primarily GitHub Copilot and ChatGPT, while emerging systems such as Gemini, Claude, and others remain underexplored.

Given this context, there is a clear need for a systematic synthesis of empirical evidence that consolidates findings, identifies methodological patterns, and highlights research gaps. This study addresses this need by conducting a Systematic Literature Review (SLR) on AI-assisted code generation tools and their impact on software development.

Specifically, this work aims to:

- Analyze the impact of AI-assisted coding tools on developer productivity
- Evaluate their effects on code quality and reliability
- Identify commonly used evaluation metrics and methodologies
- Examine reported benefits, limitations, and risks
- Highlight trends and research gaps in the field

By synthesizing evidence from 100 carefully selected studies, this paper provides a comprehensive and up-to-date view of AI-assisted software development. The findings contribute to both research and practice by offering a structured understanding of the capabilities and limitations of modern AI coding tools.

The remainder of this paper is organized as follows. Section II presents the methodology. Section III presents the results. Section IV discusses the findings. Finally, Section V concludes the paper.

II. Background

A. AI in Software Engineering

Recent advances in large language models have enabled new capabilities in program synthesis and code generation [6]. Recent advances in machine learning have enabled more interactive forms of assistance, allowing AI systems to directly support developers during coding activities.

B. AI-Assisted Programming

AI-assisted programming refers to the use of intelligent systems to support developers in writing, understanding, and maintaining code. Prior empirical studies have examined how developers interact with such tools in practice [7]. These tools can generate code snippets, suggest completions, and assist in debugging tasks, often based on natural language input.

C. Large Language Models for Code

Large language models trained on code corpora have significantly expanded the capabilities of AI-assisted programming tools [6]. These models can capture both syntactic and semantic patterns, enabling them to perform tasks such as code generation, translation, and explanation. However, prior research has shown that such models may produce insecure code, highlighting important limitations related to software security [3].

D. Evaluation Dimensions: Productivity and Code Quality

The impact of AI-assisted programming is commonly evaluated along two primary dimensions: productivity and code quality. Productivity is typically measured in terms of development time and task completion rates, while code quality includes correctness, maintainability, and security. Empirical studies often operationalize code quality through metrics such as correctness and code complexity [2]. Although productivity improvements are frequently reported, their relationship with code quality remains complex and not fully understood.

Seminal work has established functional correctness as a primary evaluation criterion for AI-generated code, typically assessed through unit tests using metrics such as pass@k [6].

III. Methodology

A. Research Design

This study follows a Systematic Literature Review (SLR) methodology, structured according to established evidence-based software engineering guidelines for systematic literature reviews [8] and the PRISMA framework [9] to ensure transparency and reproducibility. The objective is to identify, evaluate, and synthesize empirical evidence regarding the impact of AI-assisted code generation tools on software development.

The review process was designed to ensure rigor, transparency, and reproducibility through a multi-stage protocol including search, screening, quality assessment, and synthesis.

In addition to the primary studies included in the systematic analysis, seminal and foundational works were used to support conceptual definitions and evaluation frameworks.

B. Research Questions

The study addresses the following research questions:

- RQ1: What is the impact of AI-assisted coding tools on developer productivity?
- RQ2: How do these tools affect code quality?
- RQ3: What evaluation metrics are used in empirical studies?
- RQ4: What benefits are reported?
- RQ5: What limitations and risks are identified?
- RQ6: What research methodologies are used?
- RQ7: What trends and research gaps exist?

C. Search Strategy

The search process combined automated and semi-automated approaches using academic databases and AI-assisted discovery tools. Queries were constructed to target studies related to AI-assisted programming, including tools such as GitHub Copilot, ChatGPT, Codex, Gemini, and others.

Sources included:

- arXiv
- ACM Digital Library
- IEEE Xplore
- SpringerLink
- Elsevier (ScienceDirect)

The search was restricted to publications between 2020 and 2026 to reflect the emergence of modern large language model (LLM)-based tools.

D. Use of Artificial Intelligence in the Review Process

Artificial intelligence tools were used to support parts of the literature review process, particularly for:

- Initial literature discovery and retrieval
- Relevance-based screening suggestions
- Extraction and structuring of metadata (e.g., study type, metrics, tools evaluated)

However, all critical decisions regarding inclusion, exclusion, and final selection of studies were performed and validated by the authors. AI outputs were treated as assistive recommendations rather than authoritative decisions.

This hybrid approach improves efficiency while maintaining methodological rigor and human oversight.

E. Inclusion and Exclusion Criteria

Studies were included if they met all of the following criteria:

- Empirical evaluation (quantitative, qualitative, or mixed-methods)
- Focus on AI-assisted programming or code generation
- Evaluation of tools such as Copilot, ChatGPT, Codex, Gemini, or similar
- Availability of measurable outcomes (e.g., productivity, quality, accuracy)
- Published between 2020 and 2026
- Written in English

Studies were excluded if they:

- Were opinion papers, editorials, or non-scientific content
- Lacked empirical validation
- Focused on NLP tasks unrelated to programming
- Lacked verifiable metadata (e.g., missing DOI or source)
- Represented duplicate or incomplete versions of other studies

While the primary focus was on empirical studies, a small number of high-quality secondary studies (e.g., systematic literature reviews) were retained to provide contextual insights and support the interpretation of findings.

F. Study Selection Process

The study selection process was structured following systematic review best practices and reported using the PRISMA 2020 framework [9], consisting of four phases:

1) Identification: Collection of an initial dataset of 863 studies
2) Screening: Title and abstract screening reduced the dataset to 349 studies
3) Eligibility: Full-text assessment and validation reduced the dataset to 324 studies
4) Inclusion: Final selection of 100 studies after quality assessment and balancing, of which 98 were classified as primary studies and 2 as secondary studies (systematic literature reviews used for contextual support)

G. Quality Assessment

A multi-criteria scoring model was used to evaluate study quality, including:

- Empirical rigor (e.g., experimental design, sample size)
- Venue quality (e.g., ACM, IEEE, Springer)
- Completeness of metadata
- Relevance to research questions
- Diversity of evaluated tools

Each study was assigned a composite score combining:

- Screening score (from automated tools)
- Venue-based weighting
- Methodological strength
- Metadata completeness
- Tool diversity

Final selection was not purely score-based; it also incorporated balancing across publication venues and tool coverage.

H. Data Extraction and Synthesis

For each study, the following data were extracted:

- Study type
- Tools evaluated
- Evaluation metrics
- Key findings
- Research questions addressed

The extracted data were analyzed using both quantitative aggregation (e.g., frequency and percentage analysis) and qualitative synthesis to identify patterns, trends, and research gaps. Additionally, studies were classified as either primary (empirical) or secondary (e.g., surveys or systematic reviews), allowing for a clear distinction between direct evidence and contextual support during synthesis.

I. PRISMA Flow Diagram

Figure 1 presents the study selection process.

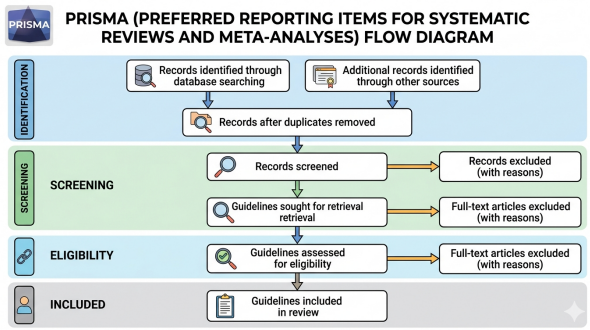

Fig. 1. PRISMA flow diagram of the study selection process

TABLE I
PRISMA STUDY SELECTION SUMMARY

Stage	Number of Studies
Initial records identified	863
After screening (title/abstract)	349
After eligibility (full-text)	324
Final included studies	100 (98 primary + 2 secondary)

J. Threats to Validity

This study is subject to several potential threats to validity, which are addressed through mitigation strategies aligned with established SLR guidelines [8].

Selection Bias. The study selection process may introduce bias due to the reliance on search queries and automated discovery tools. Relevant studies may have been omitted if they were not indexed in the selected sources or did not match the query terms. To mitigate this risk, multiple academic databases were used, and the selection process followed a structured PRISMA-based workflow.

Publication Bias. The dataset includes a significant proportion of preprints (e.g., arXiv), which may not have undergone peer review. While this reflects the rapid evolution of the field, it may also introduce bias toward preliminary findings. To

address this, publication venues were considered during the quality assessment and selection process.

Data Extraction Bias. The extraction and classification of study attributes (e.g., metrics, tools, study types) may be subject to interpretation bias. To reduce this risk, structured extraction criteria were used, and ambiguous cases were reviewed carefully.

AI-Assisted Processing Bias. Artificial intelligence tools were used to support literature discovery, screening, and data structuring. While these tools improve efficiency, they may introduce systematic biases or inconsistencies. To mitigate this threat, all critical decisions regarding inclusion, exclusion, and final selection were validated by the authors, ensuring human oversight.

Heterogeneity of Studies. The included studies vary widely in terms of methodologies, datasets, evaluation metrics, and experimental settings. This heterogeneity limits direct comparability and reduces the possibility of quantitative meta-analysis.

Temporal Bias. Given the rapid evolution of AI-assisted coding tools, the findings may become outdated as new tools and models emerge. The study mitigates this risk by focusing on recent publications (2021–2026), but this also limits historical perspective.

Reporting Bias. Some studies may selectively report positive outcomes, particularly regarding productivity improvements. This may lead to an overestimation of benefits.

Overall, while these threats cannot be fully eliminated, the use of a structured methodology, transparent criteria, and human validation helps ensure the robustness and reliability of the findings.

K. Ethical Considerations

The use of AI-assisted tools in this study complies with current ethical guidelines for research transparency. AI systems were used exclusively as support tools and did not replace human judgment in any stage of the review process.

IV. Results

A. Overview of the Selected Studies

After a multi-stage screening and validation process, a total of 100 studies were included in the final synthesis, comprising 98 primary studies and 2 secondary studies used for contextual and methodological support. The temporal distribution of the studies shows a strong concentration in recent years, with the majority published between 2024 (approximately 45%) and 2025 (approximately 42%), reflecting the rapid evolution of AI-assisted programming tools.

Regarding publication sources, the dataset includes a balanced mix of arXiv preprints (49%), ACM venues (29%), IEEE venues (13%), and other verified publishers (approximately 9%). This distribution indicates that, while the field is highly active, it is still partially consolidating in top-tier archival venues. These observations are derived from the selected studies included in this review..

B. Research Methodologies (RQ6)

The most common research methodologies identified were controlled experiments (34%), observational studies (26%), and mixed-methods approaches (10%). Case studies and surveys together account for approximately 7% of the selected studies.

These results suggest that the field is increasingly grounded in empirical evaluation, with a strong emphasis on experimental validation, consistent with trends observed in empirical software engineering research.

C. Tool Coverage and Ecosystem (RQ1, RQ4)

The literature is dominated by a small set of widely adopted tools, particularly GitHub Copilot (44%) and ChatGPT (32%). However, the dataset also includes emerging tools such as Gemini (7%), Claude and Claude Code (7%), OpenAI Codex (7%), Amazon CodeWhisperer (6%), and Tabnine (6%), as well as additional tools including Cursor, Codeium, and DeepSeek-Coder.

This distribution highlights a transition from single-tool evaluations toward a more diverse ecosystem of AI-assisted development tools, as also reflected in prior comparative evaluations of multiple AI-assisted coding tools [4].

D. Evaluation Metrics (RQ3)

The most frequently used evaluation metrics include code quality (50%), developer productivity or time savings (38%), security and vulnerability analysis (30%), accuracy and pass rates (20%), and usability or developer experience (14%).

These findings, derived primarily from empirical studies and supported by broader literature reviews, indicate that research has moved beyond productivity-centric evaluation to include multidimensional code quality assessments, alongside emerging security and risk considerations [3], [4].

Recent work has also introduced specialized metrics for evaluating bias in generated code, such as the Code Bias Score (CBS) and its multi-run variants, which capture both the prevalence and consistency of biased behaviors across model executions [10].

To provide a structured overview of how key claims in the literature are supported by empirical evidence, Table II summarizes the relationship between major findings, the type of evidence used, and the studies that support them. This mapping highlights the extent to which different dimensions—such as productivity, code quality, security, and developer experience—are grounded in empirical research.

E. Impact on Productivity (RQ1)

A substantial portion of the literature reports significant productivity improvements, particularly in tasks such as code completion, boilerplate generation, documentation, unit test generation, debugging, and other repetitive activities.

Reported gains commonly range between 30% and 50% time reduction, although results vary depending on task complexity and developer expertise, although results vary depending on developer characteristics such as experience and workload [1].

TABLE II
SUMMARY OF EVIDENCE SUPPORTING KEY CLAIMS IN AI-ASSISTED PROGRAMMING RESEARCH

Strength refers to the level of empirical support provided by the cited studies.

Claim	Type of Evidence	Supporting Studies	Strength
AI-assisted tools improve developer productivity	Controlled experiment (causal)	Peng et al.	Strong
Productivity gains vary by developer characteristics	Experimental (heterogeneous effects)	Peng et al.	Strong
AI-generated code is often correct or partially correct	Empirical code evaluation	Nguyen & Nadi	Strong
Effectiveness varies across languages and problems	Experimental evaluation	Nguyen & Nadi	Strong
AI-generated code may contain security vulnerabilities	Security analysis	Pearce et al.	Strong
Security risks require human validation	Empirical + inference	Pearce et al.	Strong
Performance varies across AI tools	Comparative empirical studies	Yetiştiren et al.; Idrisov & Schlippe	Strong
Code quality is inherently multidimensional	Empirical + conceptual	Yetiştiren et al.	Strong
Correctness is measured via unit tests (pass@k)	Foundational methodology	Chen et al.	Strong
Multiple sampling improves correctness outcomes	Experimental evaluation	Chen et al.	Strong
AI-generated code is often understandable but imperfect	Complexity analysis	Nguyen & Nadi	Strong
Productivity gains are context-dependent	Cross-study synthesis	Peng et al.; Vaithilingam et al.	Strong
No consistent productivity improvement across all scenarios	Human-subject experiment	Vaithilingam et al.	Strong
Developers prefer AI tools despite limitations	User study	Vaithilingam et al.	Strong
AI-generated code can be difficult to understand and debug	UX empirical study	Vaithilingam et al.	Strong
Over-reliance on AI suggestions is a risk	Behavioral observation	Vaithilingam et al.	Strong
AI tools function as starting points rather than final solutions	UX + empirical evidence	Vaithilingam et al.; Nguyen & Nadi	Strong
Research employs diverse evaluation metrics	SLR synthesis	This study; Yetiştiren et al.	Moderate
The field is increasingly empirical	SLR synthesis	This study	Moderate
The AI tool ecosystem is expanding	Descriptive synthesis	This study	Moderate
Relationship between productivity and quality remains unclear	Cross-study synthesis	Peng et al.; Nguyen & Nadi; Vaithilingam et al.	Strong
AI struggles with complex programming tasks	Experimental + synthesis	Chen et al.; Nguyen & Nadi; Idrisov & Schlippe	Strong
Maintainability varies across tools	Empirical evaluation	Yetiştiren et al.; Idrisov & Schlippe	Strong
Efficiency metrics are part of evaluation frameworks	Empirical evaluation	Idrisov & Schlippe	Strong

However, not all empirical evidence consistently supports these improvements. Controlled user studies have found no statistically significant differences in task completion time when using AI-assisted tools, particularly when generated code requires substantial debugging and validation [7]. Similarly, controlled studies of natural language-to-code assistants integrated within development environments report no statistically significant improvements in task completion time or correctness, despite positive user perceptions of the tools [11].

Recent randomized controlled trials conducted in real-world development settings provide strong evidence that AI assistance does not universally improve productivity. In particular, experienced developers working on real tasks in large open-source repositories were found to take approximately 19% more time when using AI tools, despite expecting substantial speed improvements [12].

F. Reported Benefits (RQ4)

Beyond productivity improvements, the literature reports several additional benefits associated with AI-assisted programming tools. These include enhanced support for repetitive tasks such as code completion and documentation, improved accessibility for novice developers, and assistance in learning new programming concepts. Empirical studies also report that developers perceive AI-assisted tools as helpful and enjoyable to use, even in cases where objective performance improvements are not observed [11].

AI tools are also frequently used to accelerate prototyping and reduce cognitive effort in routine coding activities. Additionally, developers report that such tools can serve as effective starting points for problem-solving, helping to generate initial code structures that can later be refined.

These benefits highlight the broader role of AI-assisted tools as supportive instruments that augment developer capabilities beyond simple efficiency gains.

G. Code Quality and Reliability (RQ2)

Despite productivity gains, the literature consistently identifies mixed effects on code quality. Positive outcomes are typically observed for small and well-defined tasks, particularly in terms of syntactic correctness. However, challenges arise in complex logic, multi-file contexts, and long-term maintainability. Empirical user studies further indicate that difficulties in understanding and debugging AI-generated code can negatively affect task effectiveness, particularly when developers must modify or repair code they did not originally write [7]. Additionally, empirical evaluations of in-IDE code generation and retrieval systems indicate that program correctness does not significantly improve with the use of such tools, further highlighting limitations in current AI-assisted development approaches [11].

These findings are consistent with prior evaluations showing variability in correctness and maintainability across AI-generated code [4].

H. Risks and Limitations (RQ5)

The most frequently reported risks include security vulnerabilities in generated code, hallucinated or incorrect logic, over-reliance by developers, reduced understanding of underlying code, and inconsistent performance across programming languages and domains. Security-related concerns have been clearly documented in prior empirical work on AI-generated code [3]. Recent empirical studies conducted in real-world development environments further confirm that AI-generated code frequently contains security vulnerabilities. Large-scale analyses of code snippets from GitHub projects report that approximately 25–30% of AI-generated code exhibits security weaknesses across multiple CWE categories, including high-severity vulnerabilities [13].

In addition to security risks, emerging research highlights the presence of bias in AI-generated code across sensitive attributes such as age, gender, and region, with some models exhibiting biased behavior in a substantial proportion of generated outputs [10]. These findings expand the scope of risks associated with AI-assisted programming to include ethical and fairness considerations.

Notably, RQ5 is the most represented research question (approximately 73%), indicating strong concern in the research community regarding safe and responsible adoption.

I. Research Gaps and Trends (RQ7)

The analysis reveals several key gaps in the literature, including the lack of standardized evaluation frameworks, limited longitudinal studies, underrepresentation of emerging tools such as Claude and Gemini, insufficient research on team-level and organizational impact, and scarcity of studies on long-term maintainability.

Interestingly, both developers and domain experts consistently overestimate the productivity benefits of AI assistance, highlighting a significant gap between perceived and actual performance improvements [12].

These gaps suggest important opportunities for future research in AI-assisted software development.

V. Discussion

A. Productivity Gains Are Real but Context-Dependent

The results indicate that AI-assisted coding tools provide measurable productivity improvements, particularly in repetitive and well-defined tasks such as code completion, documentation, and testing [1]. However, these gains are not uniform across all development activities. The effectiveness of such tools decreases as task complexity increases, especially in scenarios involving large codebases, multi-file dependencies, or domain-specific logic. These findings suggest that conclusions drawn from controlled or synthetic benchmarks may not generalize to real-world development environments, where task complexity, contextual knowledge, and integration overhead play a critical role.

This suggests that AI tools should be conceptualized not as replacements for developers, but as productivity amplifiers whose impact depends heavily on task structure and context. In particular, empirical studies of in-IDE natural language programming assistants suggest that improvements in developer productivity are not consistently observed, even when users report positive experiences with such tools [11]. This reinforces the notion that perceived usefulness does not necessarily translate into measurable efficiency gains. Furthermore, some controlled user studies report that while AI-generated suggestions can accelerate code production, they may also introduce additional debugging overhead, resulting in no significant net gains in task completion time [7].

B. Beyond Productivity: A Multi-Dimensional Evaluation Landscape

While early discourse around AI coding assistants emphasized speed and efficiency, the current body of literature reflects a broader evaluation perspective. Code quality, security, maintainability, and developer experience are now central dimensions of analysis [4]. Recent human-centered studies highlight that developer experience plays a critical role in the effectiveness of AI-assisted tools. While developers often perceive such tools as helpful and prefer to use them, they may struggle to understand, trust, and validate the generated code, particularly in more complex tasks [7]. This distinction between subjective user satisfaction and objective performance outcomes is also evident in studies of natural language-to-code systems, where developers express positive perceptions of the tools despite the absence of measurable improvements in productivity or correctness [11]. The discrepancy between

perceived usefulness and actual productivity outcomes further reinforces the importance of distinguishing between subjective user experience and objective performance metrics.

In addition to traditional dimensions such as productivity and code quality, recent research highlights fairness as a critical evaluation dimension in AI-assisted programming. Empirical evidence shows that generated code may embed social biases, raising ethical concerns about the deployment of such systems in real-world applications [10]. Furthermore, studies indicate that simple prompt engineering techniques are often insufficient to mitigate bias, while feedback-driven approaches can significantly reduce biased behaviors in generated code [10].

This integrated perspective is further reinforced by the structured synthesis presented in Table II, which demonstrates how different claims are supported by complementary forms of empirical evidence. The table also highlights areas of convergence and divergence across studies, particularly regarding productivity outcomes, where both positive and neutral effects are reported depending on context.

C. Dominance of Leading Tools and Emergence of New Ecosystems

The literature remains dominated by GitHub Copilot and ChatGPT, which together account for the majority of empirical studies. However, the inclusion of tools such as Claude, Gemini, OpenAI Codex, CodeWhisperer, and others indicates the emergence of a more diverse ecosystem.

Despite their lower representation, these emerging tools are critical for understanding the evolution of AI-assisted development. Their inclusion helps mitigate bias toward a small number of widely studied platforms and provides a more comprehensive view of the technological landscape.

D. Maturity of the Research Field

Although the number of studies has grown rapidly, the field remains in a relatively early stage of consolidation. The high proportion of preprints suggests that research dissemination is still driven by speed rather than archival rigor.

This does not invalidate the findings, but it does imply that conclusions should be interpreted with caution. More rigorous empirical studies in top-tier venues are emerging, but further validation is still needed across diverse contexts and tools.

E. Methodological Concentration and Its Implications

The predominance of controlled experiments and observational studies indicates a strong empirical orientation. However, the lack of standardized evaluation frameworks limits comparability across studies.

This fragmentation creates challenges for meta-analysis and synthesis, as different studies employ heterogeneous metrics, datasets, and evaluation protocols. Addressing this issue represents a key opportunity for advancing the field.

Furthermore, the diversity of evaluation metrics identified in this study, as discussed in the Results section, highlights the absence of standardized measurement frameworks. This lack of consistency limits comparability across studies and complicates efforts to synthesize findings quantitatively.

These observations are consistent with broader systematic analyses of the field, which highlight structural challenges such as the lack of standardized benchmarks and heterogeneous evaluation practices across studies.

F. Implications for Practice

From a practical perspective, organizations adopting AI-assisted coding tools should consider both the benefits and limitations identified in the literature. While these tools can significantly reduce development time, they also introduce risks related to code quality and security.

Therefore, best practices should include:

- Mandatory code review for AI-generated outputs
- Integration of security analysis tools
- Training developers to critically evaluate AI suggestions

G. Implications for Research

The findings highlight several directions for future research. These include the development of standardized evaluation methodologies, increased focus on longitudinal and real-world studies, deeper exploration of team-level impacts, and expanded investigation of emerging tools.

Additionally, there is a need to better understand how AI-assisted development affects software maintainability and developer cognition over time.

H. Threats to Validity

This study is subject to several limitations. First, the dataset includes a significant proportion of preprints, which may not have undergone rigorous peer review. Second, the reliance on automated screening tools introduces potential biases in study selection and scoring. Third, the diversity of evaluation metrics across studies limits direct comparability.

Despite these limitations, the systematic filtering, validation, and balancing procedures applied in this study provide a robust foundation for synthesizing current evidence in AI-assisted software development.

VI. Conclusion

This paper presented a systematic literature review of AI-assisted programming in software engineering, focusing on the capabilities of current tools, their impact on developer productivity, and their influence on code quality. By analysing studies published between 2020 and 2026, this work provides a consolidated view of a rapidly evolving research area.

The findings indicate that AI-assisted programming tools offer substantial benefits in terms of productivity, particularly in tasks that are repetitive or well-defined. However, their impact on code quality remains less consistent. While efficiency improvements are frequently reported, they are sometimes accompanied by issues related to correctness, maintainability, and security. These results highlight the importance of balancing productivity gains with careful validation of AI-generated outputs.

Beyond summarising existing evidence, this study contributes by organising the literature into a structured framework and identifying key trends, limitations, and research gaps. In particular, the analysis underscores the need for more rigorous empirical studies, especially in real-world industrial contexts, as well as the development of standardised evaluation metrics.

Looking forward, AI-assisted programming is likely to play an increasingly central role in software development. Understanding how to effectively integrate these tools into development workflows, while mitigating their limitations, remains an important challenge for both researchers and practitioners. Continued investigation in this area will be essential to fully realise the potential of AI in software engineering.

REFERENCES

[1] S. Peng, E. Kalliamvakou, P. Cihon, and M. Demirer, "The Impact of AI on Developer Productivity: Evidence from GitHub Copilot," *arXiv preprint arXiv:2302.06590*, 2023.

[2] N. Nguyen and S. Nadi, "An empirical evaluation of github copilot's code suggestions," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022.

[3] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," *arXiv preprint arXiv:2108.09293*, 2022.

[4] B. Yetiştiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the code quality of ai-assisted code generation tools," *arXiv preprint arXiv:2304.10778*, 2023.

[5] B. Idrisov and T. Schlippe, "Program code generation with generative ais," *Algorithms*, vol. 17, no. 2, 2024.

[6] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, A. Ray, R. Puri, G. Krueger, M. Petrov, H. Khlaaf, G. Sastry, P. Mishkin, B. Chan, S. Gray, N. Ryder, M. Pavlov, A. Power, L. Kaiser, M. Bavarian, C. Winter, P. Tillet, F. P. Such, D. Cummings, M. Plappert, F. Chantzis, E. Barnes, A. Herbert-Voss, W. H. Guss, A. Nichol, A. Paino, N. Tezak, J. Tang, I. Babuschkin, S. Balaji, S. Jain, W. Saunders, C. Hesse, A. N. Carr, J. Leike, J. Achiam, V. Misra, E. Mirikawa, A. Radford, M. Knight, M. Brundage, M. Murati, K. Mayer, P. Welinder, B. McGrew, D. Amodei, S. McCandlish, I. Sutskever, and W. Zaremba, "Evaluating large language models trained on code," *arXiv*, vol. 7, no. OOPSLA1, 2021.

[7] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. Experience: Evaluating AI Code Generation Tools," in *CHI Conference on Human Factors in Computing Systems*, 2022.

[8] B. Kitchenham and S. Charters, "Guidelines for performing Systematic Literature Reviews in Software Engineering," *EBSE Technical Report*, 2007.

[9] M. Page, J. E. McKenzie, P. Bossuyt, I. Bourton, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stweart, J. Thomas, A. Tricco, V. A. Welch, P. Withing, and D. Moher, "The PRISMA 2020 statement," *BMJ*, 2021.

[10] D. Huang, J. M. Zhang, Q. Bu, X. Xie, J. Chen, and H. Cui, "Bias testing and mitigation in llm-based code generation," *ACM Transactions on Software Engineering and Methodology*, Mar. 2025. [Online]. Available: https://dl.acm.org/doi/abs/10.1145/3724117

[11] F. F. Xu, B. Vasilescu, and G. Neubig, "In-IDE Code Generation from Natural Language: Promise and Challenges," *ACM Transactions on Software Engineering and Methodology*, vol. 31, no. 2, pp. 1–47, Apr. 2022. [Online]. Available: https://dl.acm.org/doi/10.1145/3487569

[12] J. Becker, N. Rush, E. Barnes, and D. Rein, "Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity," Jul. 2025, arXiv:2507.09089 [cs]. [Online]. Available: http://arxiv.org/abs/2507.09089

[13] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen, "Security Weaknesses of Copilot-Generated Code in GitHub Projects: An Empirical Study," *ACM Transactions on Software Engineering and Methodology*, vol. 34, no. 8, pp. 1–34, Nov. 2025. [Online]. Available: https://dl.acm.org/doi/10.1145/3716848